

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2022/2023



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UCCD 1004 PROGRAMMING CONCEPTS AND PRACTICES

ASSIGNMENT 2

Group 52			
Name	ID	Programme	UTAR Email
WONG PEI KEI	2207466	CN	xlzz08@1utar.my
BERNARD GOH PEK HONG	2300075	CN	bernardgoh0912@1utar.my
ENG ZI YEE	2104143	CN	ziyee1214@1utar.my

ii. Task Division

	Name	Modules	Description	*A2 Contribution (Overall, %)	
1.	WONG PEI KEI	Add/Delete	void add(); / void mbSignup(); / bool testNum(char[], int); void dlt(); / void mbcancelacc(int, bool&);	1	100
		View	void read(); / void mbbuyticket(int, int, double&); void mbwallet(double &);	2	100
		Check in/ Check out	void mbLogin(int&); / void mbMenu();	3	100
2.	BERNARD GOH PEK HONG	Time/Date	void mbbuyticket(int, int, double&);	1	100
		Seat	void mbbuyticket(int, int, double&); void print_seating_chart(); bool reserve_seat(int, int);	2	100
		Payment	void mbbuyticket(int, int, double&); double calculate_total_cost(int, int);	3	100
3.	ENG ZI YEE	Check in	void mbLogin(int&); / void mbMenu();	1	100
		Check out	void mbLogin(int&); / void mbMenu();	2	100
		-	-	3	0

* Depends on the evaluation of the markers as well;

iii. Objective

Add / Delete module

- Admin mode (add/delete movie using txt file to save record)
- Member mode (register new user/delete current user account using txt file to save record)

View module

- Admin mode (to view movie list whether any movie add or delete from user input)
- Member mode (to view movie list whether any movie have chosen by member to buy their ticket)
- Wallet section (to view member balance whether any top up service upload or member buy ticket deducted from payment and updated)

Check in / Check out module

- Member mode (user login check whether exist or not, if exist, proceed to member menu)
- Member mode (user proceed to logout, auto logout)

Date / Time module

- Member buy ticket can choose a specific date and time

Seat module

- Member can choose seat by rows to columns. If seat reserved, it wouldn't allow to choose

Payment module

- Payment should deduct from member wallet, if wallet not enough money, auto cancel transaction. Else, proceed.
- After member made their payment, ticket will be print out, member should have their QR code to scan.

iv. Pseudocode

```
INCLUDE iostream library
INCLUDE string library
INCLUDE fstream library
INCLUDE iomanip library
INCLUDE cctype library
INCLUDE cstring library
DEFINE the CRT_SECURE_NO_WARNINGS macro
USING namespace std
```

```
DECLARE Constant Integer arraySize = 1000
DECLARE Constant Integer pwSize = 9
DECLARE Constant Integer rows = 10
DECLARE Constant Integer columns = 20
DECLARE Integer seats[rows][columns] = { 0 }
```

```
DEFINE Struct movieType
    DECLARE String name
    DECLARE Double price
```

```
DEFINE Struct memberSign
    DECLARE String name
    DECLARE Character password[pwSize]
```

```
FUNCTION Void admb()
FUNCTION Void admMenu(int&)
FUNCTION Void read()
FUNCTION Void add()
FUNCTION Void dlt()
FUNCTION Void mbMenu()
FUNCTION Void mbSignup()
FUNCTION Boolean testNum(char [], int)
FUNCTION Void mbLogin(int&)
```

```
FUNCTION Void mbLoginMenu(int&)
FUNCTION Void mbbuyticket(int, int, double&)
FUNCTION Void print_seating_chart()
FUNCTION Double calculate_total_cost(int, int)
FUNCTION Boolean reserve_seat(int, int)
FUNCTION Void mbwallet(double&)
FUNCTION Void mbcancelacc(int, bool&)
```

```
MAIN FUNCTION()
```

```
    CALL FUNCTION admb()
```

```
    READ input stream
```

```
    RETURN 0
```

```
VOID FUNCTION admb()
```

```
BEGIN
```

```
    DECLARE Integer choiceA, num
```

```
    DECLARE Boolean choice INITIALIZE to false
```

```
    DECLARE Character quitChoice
```

```
    DO
```

```
        CHANGE console color to 0d
```

```
        DISPLAY "YSSL Cinema"
```

```
        DISPLAY "1. Admin Mode"
```

```
        DISPLAY "2. Member Mode"
```

```
        DISPLAY "3. Quit"
```

```
        DISPLAY "Please input a selection: "
```

```
        READ choiceA
```

```
    IF choiceA == 1 THEN
```

```
        DO
```

```
            CLEAR console screen
```

```
            CALL FUNCTION admMenu with num
```

```
            IF num == 1 THEN
```

```
                CALL FUNCTION read()
```

again!”

```
        ELSE IF num == 2 THEN
            CALL FUNCTION add()
        ELSE IF num == 3 THEN
            CALL FUNCTION dlt()
        ELSE IF num > 4 OR num < 0 THEN
            DISPLAY “Invalid input. Please press ENTER to select
again!”

            READ input stream
            READ input stream
        ELSE IF num == 4 THEN
            INITIALIZE choice to false
        END IF

        WHILE num > 0 AND num != 4
        END WHILE

        CLEAR console screen

ELSE IF choiceA == 2 THEN
    CLEAR console screen
    CALL FUNCTION mbMenu()

ELSE IF choiceA == 3 THEN
    DISPLAY “Confirm to QUIT the system?(Y/N): “
    READ quitChoice
    WHILE quitChoice != 'Y'      AND quitChoice != 'y' AND
        quitChoice != 'n' AND quitChoice != 'N'
        DISPLAY “Invalid input. Please re-type again! (Y-Yes, N-No): “
        READ quitChoice
    END WHILE
    IF quitChoice = ‘Y’ OR quitChoice = ‘y’
        INITIALIZE choice to true
    END IF
    CLEAR console screen
ELSE
    DISPLAY “Invalid input. Please press ENTER to select again!”
```

```

        READ input stream
        READ input stream
        CLEAR console screen
    END IF
WHILE NOT choice
END WHILE
END

VOID FUNCTION mbMenu()
BEGIN
    DECLARE choiceB, a
    DECLARE choice INITIALIZE to false
    DO
        CHANGE console color 0b
        DISPLAY "New User or Exist?"
        DISPLAY "1. New Register"
        DISPLAY "2. User Login"
        DISPLAY "Please input a selection: "
        READ choiceB

        IF choiceB == 1 THEN
            CLEAR console screen
            CALL FUNCTION mbSignup()
            CLEAR console screen
            INITIALIZE choice to true

        ELSE IF choiceB == 2 THEN
            CLEAR console screen
            CALL FUNCTION mbLogin with a
            CALL FUNCTION mbLoginMenu with a
            INITIALIZE choice to true

        ELSE
            DISPLAY "Invalid input. Please press ENTER to select again!"

```

```

        READ input stream
        READ input stream
        CLEAR console screen
    END IF
    WHILE NOT choice
    END WHILE
END

VOID FUNCTION mbSignup()
BEGIN
    DECLARE Integer i INITIALIZE to 0
    DECLARE Boolean valid INITIALIZE to false
    DECLARE memberSign member[arraySize]
    OPENFILE Outfile "member.txt" for WRITE and APPEND
    OPENFILE Infile "member.txt" for READ and APPEND

    WHILE NOT EOF Infile
        READFILE Infile, member[i].name
        READFILE Infile, member[i].password, pwSize
        i++
    END WHILE
    CLOSEFILE Infile

    CHANGE console color to 0b
    DISPLAY "Welcome To YSLL Cinema!"
    DISPLAY "New Member Sign Up"
    DISPLAY "Please ensure that you set the PASSWORD should be memorable!" SET red
color
    DISPLAY a new line

    DECLARE Boolean usernamevalid to false
    DECLARE Integer a

    DISPLAY "Username: "

```


READ IGNORE newline character

READ member[i].name

DO

FOR every a in i

IF member[i].name != member[a].name

DISPLAY member[i].name

INITIALIZE usernamevalid to true

FOR every a in I

IF member[i].name == member[a].name

DISPLAY "This username has been created by other user."

DISPLAY "Please re-type other username!"

DISPLAY new line

DISPLAY "Username: "

READ member[i].name

READ IGNORE newline character

INITIALIZE usernamevalid to false

WHILE NOT usernamevalid

WRITEFILE Outfile, new line

WRITEFILE Outfile, member[i].name, end line

DISPLAY a new line

DO

DISPLAY "L-Letter; N-Number" SET blue color

DISPLAY "(Format - LLLLNNNN)" SET blue color

DISPLAY "Password: "

READ member[i].password, pwSize

IF CALL FUNCTION testNum with member[i].password, pwSize THEN

WRITEFILE Outfile, member[i].password

DISPLAY a new line

DISPLAY "Member successfully created! Your member ID is "

DISPLAY "Press ENTER to return back to menu: "

READ input stream

INITIALIZE valid to true

ELSE

DISPLAY a new line

DISPLAY "Format incorrect! Please set the PASSWORD again."

END IF

WHILE NOT valid

END WHILE

CLOSEFILE Outfile

END

BOOLEAN FUNCTION testNum with CHAR memberpw[], INT pwSize

BEGIN

DECLARE Integer count

FOR the first 4 counts in the password

IF each count of memberpw NOT alphabet

RETURN false

END FOR

FOR the last 4 counts in the password

IF each count of memberpw NOT digit

RETURN false

END FOR

RETURN true

END

VOID FUNCTION mbLogin RETURNING a

BEGIN

INITIALIZE a to 0

```

DECLARE Integer i to 0, lgno to 0
DECLARE Boolean valid to false
DECLARE memberSign member[arraySize], login[arraySize]
OPENFILE Infile "member.txt" for READ and APPEND

WHILE NOT EOF Infile
    READFILE Infile, member[i].name
    READFILE Infile, member[i].password, pwSize
    i++
END WHILE
CLOSEFILE Infile

DO
    CHANGE console color to 0b
    DISPLAY "Welcome To YSLL Cinema!"
    DISPLAY "Member Sign In"
    DISPLAY a new line
    READ IGNORE newline character
    READ loginmb[lgno].name
    DISPLAY a new line
    DISPLAY "Password: "
    READ loginmb[lgno].password, pwSize

    FOR every count 'a' read in member name and password
        IF member[a].name == loginmb[lgno].name
            AND member[a].password COMPARED with loginmb[lgno].password == 0
                DISPLAY "You're successfully log in! Press ENTER to MEMBER
MENU."

                READ input stream
                CLEAR console screen
                INITIALIZE valid to true
                BREAK FOR LOOP
        END IF
    END FOR
END DO

```

```

        IF member[a].name != loginmb[lgno].name
        OR member[a].password COMPARED with loginmb[lgno].password == -1
            DISPLAY "Not existing account, press ENTER to sign in again."
            READ input stream
            CLEAR console screen
        END IF
    END FOR
WHILE NOT valid
END

```

```

VOID FUNCTION mbLoginMenu RETURNING a
BEGIN
    DECLARE Integer choiceC, current_idno to a
    DECLARE Double balance to 0
    DECLARE movieType movie[arraySize]
    DECLARE Boolean loginChoice to false, auto_logout to false, buyticketChoice to false

    DO
        CHANGE console color to 0b
        DISPLAY "YSSL Cinema - Member Login Menu"
        DISPLAY "1. Movie List"
        DISPLAY "2. Buy Ticket"
        DISPLAY "3. Wallet"
        DISPLAY "4. Delete Acoount"
        DISPLAY "5. Logout"
        DISPLAY "Please input a selection: "
        READ choiceC
        DISPLAY new line

        IF choiceC == 1 THEN
            CALL FUNCTION read()

        ELSE IF choiceC == 2 THEN
            DO

```

```

CLEAR console screen
OPENFILE Infile "cinema.txt" for READ and APPEND
DECLARE Integer i to 0, movieNo

DISPLAY "1. Buy Ticket"
DISPLAY "Movie List"
DISPLAY "Name Price"
IF Infile is OPEN
    WHILE NOT EOF Infile
        READFILE Infile, movie[i].name
        READFILE Infile, movie[i].price
        DISPLAY i+1, ".", movie[i].name
        DISPLAY FIXED, SETPRECISION to 2
        DISPLAY "RM", movie[i].price
        i++
        READFILE IGNORE newline character
    END WHILE
END IF
CLOSEFILE Infile

DISPLAY "Choose a movie: "
READ movieNo

IF movieNo <= I THEN
    CALL FUNCTION mbbuyticket with movieNo, i, balance
    CLEAR console screen
    INITIALIZE buyticketChoice to true

ELSE
    DISPLAY "Invalid input. Press ENTER to re-select again!"
    READ input stream
    READ input stream (let user press ENTER)
END IF
WHILE NOT buyticketChoice

```

```

        END WHILE

    ELSE IF choiceC == 3 THEN
        CALL FUNCTION mbwallet with balance
        CLEAR console screen

    ELSE IF choiceC == 4 THEN
        CALL FUNCTION mbcancelacc with current_idno, auto_logout
        CLEAR console screen
        IF auto_logout is true
            INITIALIZE loginChoice to true
        END IF

    ELSE IF choiceC == 5 THEN
        DISPLAY "Thank for your support to our YSLL Cinema!"
        DISPLAY "Press ENTER to proceed logout."
        READ input stream
        READ input stream (let user press ENTER)
        CLEAR console screen
        INITIALIZE loginChoice to true
    END IF

    WHILE NOT loginChoice
    END WHILE
END

VOID FUNCTION mbbuyticket with Integer movieNo, i RETURNING Double balance
BEGIN
    DECLARE movieType movie[arraySize]
    CLEAR console screen
    CHANGE console color to 0d

    DECLARE Integer date, time, numSeats = 0
    DECLARE Double payment, totalPayment, change
    DECLARE Character choice

```

```
WHILE true
    DISPLAY "Date Available"
    DISPLAY "1. 16/4/2023"
    DISPLAY "2. 17/4/2023"
    DISPLAY "3. 18/4/2023"
    DISPLAY "4. 19/4/2023"
    DISPLAY "5. 20/4/2023"
    DISPLAY "Select a date: "
    READ date

    IF date >= 1 AND date <= 5 THEN
        BREAK WHILE
    ELSE
        DISPLAY "Please enter a number between 1 to 5 only"
    END IF
END WHILE
DISPLAY new line
CLEAR console screen
```

```
WHILE true
    DISPLAY "You have selected the date", date, "."
    SWITCH date
    CASE 1
        DISPLAY "16/4/2023"
        BREAK SWITCH
    CASE 2
        DISPLAY "17/4/2023"
        BREAK SWITCH
    CASE 3
        DISPLAY "18/4/2023"
        BREAK SWITCH
    CASE 4
        DISPLAY "19/4/2023"
```

```

        BREAK SWITCH
CASE 5
    DISPLAY "20/4/2023"
    BREAK SWITCH
END CASE

DISPLAY new line
DISPLAY "Do you want to change your selection? (Y/N): "
DECLARE Character choice
READ choice
CLEAR console screen

IF choice == 'Y' OR choice == 'y' THEN
    WHILE true
        DISPLAY "Date Available"
        DISPLAY "1. 16/4/2023"
        DISPLAY "2. 17/4/2023"
        DISPLAY "3. 18/4/2023"
        DISPLAY "4. 19/4/2023"
        DISPLAY "5. 20/4/2023"
        DISPLAY "Select a date: "
        READ date

        IF date >= 1 && date <= 5 THEN
            BREAK WHILE LOOP
        ELSE
            DISPLAY "Please enter a number between 1 to 5 only"
        END IF
        DISPLAY new line
        CLEAR console screen
    END WHILE

ELSE IF    choice == 'N' OR choice == 'n' THEN
    BREAK WHILE

```



```
        ELSE
            DISPLAY "Invalid input. Please enter Y or N only."
        END IF
    END WHILE

    DISPLAY "You have selected date", date, "."
    SWITCH date
    CASE 1
        DISPLAY "16/4/2023"
        BREAK SWITCH
    CASE 2
        DISPLAY "17/4/2023"
        BREAK SWITCH
    CASE 3
        DISPLAY "18/4/2023"
        BREAK SWITCH
    CASE 4
        DISPLAY "19/4/2023"
        BREAK SWITCH
    CASE 5
        DISPLAY "20/4/2023"
        BREAK SWITCH
    END CASE

    DISPLAY new line
    CLEAR console screen

    CHANGE console color to 0e
    WHILE true
        DISPLAY "Time available"
        DISPLAY "1. 10:00AM"
        DISPLAY "2. 01:15PM"
        DISPLAY "3. 01:45PM"
        DISPLAY "4. 03:45PM"
```

```

    DISPLAY "5. 04:15PM"
    DISPLAY "6. 06:45PM"
    DISPLAY "7. 09:15PM"
    DISPLAY "Select a time: "
    READ time

    IF time >= 1 AND time <= 7 THEN
        BREAK WHILE
    ELSE
        DISPLAY "Please enter a number between 1 to 7 only"
    END IF
    DISPLAY new line
    CLEAR console screen
    END WHILE

    WHILE true
        DISPLAY "You have selected the time", time, "."
        SWITCH time
        CASE 1
            DISPLAY "10:00AM"
            BREAK SWITCH
        CASE 2
            DISPLAY "01:15PM"
            BREAK SWITCH
        CASE 3
            DISPLAY "01:45PM"
            BREAK SWITCH
        CASE 4
            DISPLAY "03:45PM"
            BREAK SWITCH
        CASE 5
            DISPLAY "04:15PM"
            BREAK SWITCH
        CASE 6:

```

```

        DISPLAY "06:45PM"
        BREAK SWITCH
CASE 7
        DISPLAY "09:15PM"
        BREAK SWITCH
END CASE

DISPLAY new line
DISPLAY "Do you want to change your selection? (Y/N): "
DECLARE Character choice
READ choice
CLEAR console screen

IF choice == 'Y' OR choice == 'y' THEN
    WHILE true
        DISPLAY "Time available"
        DISPLAY "1. 10:00AM"
        DISPLAY "2. 01:15PM"
        DISPLAY "3. 01:45PM"
        DISPLAY "4. 03:45PM"
        DISPLAY "5. 04:15PM"
        DISPLAY "6. 06:45PM"
        DISPLAY "7. 09:15PM"
        DISPLAY "Select a time: "
        READ time

        IF time >= 1 AND time <= 7 THEN
            BREAK WHILE
        ELSE
            DISPLAY "Please enter a number between 1 to 7 only"
        END IF
    END WHILE
    DISPLAY new line
    CLEAR console screen

```

```

        ELSE IF choice == 'N' OR choice == 'n' THEN
            BREAK WHILE

        ELSE
            DISPLAY "Invalid input. Please enter Y or N only."
        END IF
    END WHILE

    DISPLAY "You have selected time", time, "."
    SWITCH time
    CASE 1
        DISPLAY "10:00AM"
        BREAK SWITCH
    CASE 2
        DISPLAY "01:15PM"
        BREAK SWITCH
    CASE 3
        DISPLAY "01:45PM"
        BREAK SWITCH
    CASE 4
        DISPLAY "03:45PM"
        BREAK SWITCH
    CASE 5
        DISPLAY "04:15PM"
        BREAK SWITCH
    CASE 6:
        DISPLAY "06:45PM"
        BREAK SWITCH
    CASE 7
        DISPLAY "09:15PM"
        BREAK SWITCH
    END CASE
    DISPLAY new line

```

CLEAR console screen

DISPLAY new line

DECLARE Character choice to 'y'

DO

CHANGE console color to 0e

DISPLAY "SCREEN"

CALL FUNCTION print_seating_chart()

DISPLAY new line

DISPLAY "Enter column number (1-", columns, "): "

DECLARE Integer seat

READ seat

DISPLAY "Enter row number (1-", rows, "): "

DECLARE Integer row

READ row

IF row < 1 OR row > rows OR seat < 1 OR seat > columns THEN

DISPLAY "Invalid input. Press ENTER to select again."

READ input stream

READ input stream (let user press ENTER)

CLEAR console screen

INITIALIZE choice to 'y'

CONTINUE WHILE

END IF

DECLARE Boolean reserved to CALL FUNCTION reserve_seat with row, seat

IF reserved is true THEN

DISPLAY "Seat reserved."

numSeats++

ELSE

DISPLAY "Seat is already reserved. Press ENTER to select again!"

DISPLAY new line

READ input stream

```

        READ input stream (let user press ENTER)
        CLEAR console screen
        CONTINUE WHILE
    END IF

    DISPLAY "Do you want to reserve more seats? (Y/N): "
    READ choice
    CLEAR console screen
    WHILE choice == 'Y' OR choie == 'y'
    END WHILE

    INITIALIZE totalPayment to CALL FUNCTION calculate_total_cost with numSeats,
movieN

    DISPLAY newline, newline
    DISPLAY "Total price is: RM", totalPayment
    DISPLAY "Wallet balance is: RM", balance

    CALCULATE change = balance - totalPayment
    DECLARE Boolean payment_successful to true

    IF change >= 0 THEN
        INITIALIZE balance to change
        DISPLAY "Change: RM", change
        DISPLAY "Thank you for purchasing and have a good day :)"
        DISPLAY new line

    ELSE
        DISPLAY "Error: Insufficient amount."
        DISPLAY "Payment Unsuccessful, please top up WALLET before transaction"
        DISPLAY "Press ENTER return back to Member Menu."
        READ input stream
        READ input stream (let user press ENTER)
        INITIALIZE payment_successful to false
    
```

END IF

DISPLAY new line

WHILE payment_successful is true

 DISPLAY “YSSL Cinema Receipt”

 DISPLAY “Movie Title: ”

 OPENFILE Infile “cinema.txt” for READ

 WHILE NOT EOF Infile

 FOR every a in movie list

 READFILE Infile, movie[a].name

 READFILE Infile, movie[a].price

 READFILE IGNORE newline character

 END FOR

 END WHILE

 CLOSEFILE Infile

 DISPLAY movie[movieNo - 1].name

 DISPLAY new line

 DISPLAY “Movie Date: ”

 SWITCH date

 CASE 1

 DISPLAY “16/4/2023”

 BREAK SWITCH

 CASE 2

 DISPLAY “17/4/2023”

 BREAK SWITCH

 CASE 3

 DISPLAY “18/4/2023”

 BREAK SWITCH

 CASE 4

 DISPLAY “19/4/2023”

 BREAK SWITCH

 CASE 5

DISPLAY "20/4/2023"

BREAK SWITCH

END CASE

DISPLAY new line

DISPLAY "Movie Time: "

SWITCH time

CASE 1

DISPLAY "10:00AM"

BREAK SWITCH

CASE 2

DISPLAY "01:15PM"

BREAK SWITCH

CASE 3

DISPLAY "01:45PM"

BREAK SWITCH

CASE 4

DISPLAY "03:45PM"

BREAK SWITCH

CASE 5

DISPLAY "04:15PM"

BREAK SWITCH

CASE 6:

DISPLAY "06:45PM"

BREAK SWITCH

CASE 7

DISPLAY "09:15PM"

BREAK SWITCH

END CASE

DISPLAY new line

DISPLAY "Total payment: RM", totalPayment

DISPLAY "Show this ticket to the counter to clarify"


```
    DISPLAY “the ticket validity due to security reason”
    DISPLAY “then only assign according to the seating”
    DISPLAY “position. Sorry for problem caused.”
    DISPLAY “Have a great day! :) QR CODE”
```

```
    READ input stream
    READ input stream (let user press any key)
    INITIALIZE payment_successful to false
END WHILE
END
```

```
VOID FUNCTION print_seating_chart()
BEGIN
    DISPLAY new line
    FOR every Integer i in rows
        FOR every Integer j in columns
            IF seats[i][j] == 0 THEN
                DISPLAY “0”
            ELSE
                DISPLAY “X”
            END IF
        END FOR
    END FOR
    DISPLAY new line
END
```

```
DOUBLE FUNCTION calculate_total_cost with Integer numSeats, movieNo
BEGIN
    DECLARE movieType movie[arraySize]
    OPENFILE Infile “cinema.txt” for READ
    DECLARE Integer i = 0
    WHILE NOT EOF Infile
        READFILE IGNORE newline character
        READFILE Infile, movie[i].name
```

```

        READFILE Infile, movie[i].price
        i++
    END WHILE
    RETURN numSeats * movie[movieNo - 1].price
END

```

```

BOOLEAN FUNCTION reserve_seat with Integer row, seat
BEGIN
    IF seats[row-1][seat-1] == 0 THEN
        seats[row-1][seat-1] = 1
        RETURN true
    ELSE
        RETURN false
    END IF
END

```

```

VOID FUNCTION mbwallet RETURNING balance
BEGIN
    DECLARE Integer topup
    DECLARE Character topupChoice
    DECLARE Boolean topupexit to false

    CLEAR console screen
    DISPLAY "Wallet"
    DISPLAY FIXED, SETPRECISION to 2
    DISPLAY "Blance = RM", balance
    DISPLAY "Would you like to top up your wallet? (Y/N): "
    READ topupChoice
    WHILE topupChoice != 'Y' AND topupChoice != 'y'
        AND topupChoice != 'N' AND topupChoice != 'n'
            DISPLAY "Invalid input. Please re-type again! (Y-Yes, N-No): "
            READ topupChoice
        END WHILE

```

```
IF topupChoice == 'Y' OR topupChoice == 'y' THEN
    CLEAR console screen
    DISPLAY "Top Up Service"
    DISPLAY "RM10"
    DISPLAY "RM20"
    DISPLAY "RM50"
    DISPLAY "RM100"

    DO
        DISPLAY new line
        DISPLAY "Please key in a value to TOPUP: "
        READ topup

        SWITCH topup
        CASE 10
            CALCULATE balance += topup
            BREAK SWITCH
        CASE 20
            CALCULATE balance += topup
            BREAK SWITCH
        CASE 50
            CALCULATE balance += topup
            BREAK SWITCH
        CASE 100
            CALCULATE balance += topup
            BREAK SWITCH
        DEFAULT
            DISPLAY "Invalid input. Please re-type TOPUP value!"
            CONTINUE WHILE
        END CASE

    DISPLAY "Would you like to TOP UP again? (Y/N): "
    READ topupChoice
```

```

        WHILE topupChoice != 'Y' AND topupChoice != 'y'
        AND topupChoice != 'N' AND topupChoice != 'n'
            DISPLAY "Invalid input. Please re-type again! (Y-Yes, N-No): "
            READ topupChoice
        END WHILE

        IF topupChoice == 'N' OR topupChoice == 'n'
            DISPLAY "Press ENTER to return back to Member MENU."
            READ input stream
            READ input stream (let user to press ENTER)
            INITIALIZE topupexit to true
        END IF

        WHILE NOT topupexit
        END WHILE

    ELSE IF topupChoice == 'N' OR topupChoice == 'n' THEN
        DISPLAY "Press ENTER to return back to Member MENU."
        READ input stream
        READ input stream (let user to press ENTER)
    END IF
END

VOID FUNCTION mbcancelacc with Integer current_idno RETURN Boolean auto_logout
BEGIN
    DECLARE Integer i = 0
    DECLARE Character deleteChoice
    DECLARE memberSign member[arraySize]
    OPENFILE Infile "member.txt" for READ and APPEND

    WHILE NOT EOF Infile
        READFILE Infile, member[i].name
        READFILE Infile, member[i].password, pwSize
        i++
    END WHILE

```

CLOSEFILE Infile

DISPLAY "Do you want to delete your current account? (Y/N): "

READ deleteChoice

WHILE deleteChoice != 'Y' AND deleteChoice != 'y'

AND deleteChoice != 'N' AND deleteChoice != 'n'

 DISPLAY "Invalid input. Please re-type again! (Y-Yes, N-No): "

 READ deleteChoice

END WHILE

IF deleteChoice == 'Y' OR deleteChoiec == 'y'

 OPENFILE Outfile "member.txt" for TRUNCATE

 FOR every Integer a in i-1 (total member)

 IF member[a].name == member[current_idno].name AND

 member[a].password COMPARED member[current_idno].password == 0

 INITIALIZE member[a].name to member[a+1].name

 member[a].password COPY value to member[a+1].password

 IF a != 0 THEN

 WRITEFILE Outfile, new line

 END IF

 WRITEFILE Outfile, member[a].name

 WRITEFILE Outfile, member[a].password

 ELSE IF member[a].name != member[current_idno].name OR

 member[a].password COMPARED member[current_idno].password == -1

 IF a != 0 THEN

 WRITEFILE Outfile, new line

 END IF

 WRITEFILE Outfile, member[a].name

 WRITEFILE Outfile, member[a].password

 END IF

END FOR

```
CLOSEFILE Outfile
DISPLAY "Account deleted. Press ENTER auto logout..."
READ input stream
READ input stream (let user to press ENTER)
INITIALIZE auto_logout to true
```

```
ELSE IF deleteChoice == 'N' OR deleteChoice == 'n'
    DISPLAY "Press ENTER return back to Member MENU."
    READ input stream
    READ input stream (let user to press ENTER)
```

```
END IF
END
```

```
VOID FUNCTION admMenu RETURN Integer choice
```

```
BEGIN
    CHANGE console color to 0a
    DISPLAY "YSSL Cinema - Admin Operating Menu"
    DISPLAY "1. Movie List"
    DISPLAY "2. Add Movie"
    DISPLAY "3. Delete Movie"
    DISPLAY "4. Back"
    DISPLAY "Please input a selection: "
    READ choice
```

```
END
```

```
VOID FUNCTION read()
```

```
BEGIN
    OPENFILE Infile "cinema.txt" for READ and APPEND
    DECLARE movieType movie[arraySize]
    DECLARE Integer i to 0

    CLEAR console screen
    DISPLAY "Movie List"
```

DISPLAY "Name Price"

IF Infile is open THEN

 WHILE NOT EOF Infile

 READFILE Infile, movie[i].name

 READFILE Infile, movie[i].price

 DISPLAY i+1, ":", movie[i].name

 DISPLAY FIXED, SETPRECISION to 2

 DISPLAY "RM", movie[i].price

 i++

 READFILE IGNORE newline character

 END WHILE

 CLOSEFILE Infile

ELSE

 DISPLAY "Unable to open storing file!"

END IF

DISPLAY "Press ENTER to return back to Operating Menu."

READ input stream

READ input stream (let user to press ENTER)

CLEAR console screen

END

VOID FUNCTION add()

BEGIN

 OPENFILE Infile "cinema.txt" for READ and APPEND

 OPENFILE Outfile "cinema.txt" for WRITE and APPEND

 DECLARE movieType movie[arraySize]

 DECLARE Character addMovie

DO

 DECLARE Integer i to 0

 WHILE NOT EOF Infile

```
        READFILE Infile, movie[i].name
        READFILE Infile, movie[i].price
        i++
        READFILE IGNORE newline character
    END WHILE
```

```
    DISPLAY new line
    DISPLAY "Add Movie"
    DISPLAY "Name: "
    WRITEFILE new line
    READ IGNORE newline character
    READ movie[i].name
    WRITEFILE Outfile, movie[i].name
    DISPLAY "Price: "
    READ movie[i].price
    WRITEFILE Outfile, movie[i].price
```

```
    DISPLAY "Do you want to add more movie?(Y/N): "
    READ addMovie
```

```
    WHILE addMovie != 'Y' AND addMovie != 'y'
    AND addMovie != 'N' AND addMovie != 'n'
        DISPLAY "Invalid input. Please type again (Y-Yes, N-No): "
        READ addMovie
    END WHILE
```

```
    WHILE addMovie == 'Y' OR addMovie == 'y'
    END WHILE
    CLOSEFILE Infile
    CLOSEFILE Outfile
```

```
    DISPLAY new line
    DISPLAY "Press ENTER to return back to Operating Menu."
    READ input stream
```



```

    READ input stream (let user to press ENTER)
    CLEAR console screen
END

VOID FUNCTION dlt()
BEGIN
    DECLARE movieType movie[arraySize]
    DECLARE Integer i, a to 0, dlno
    DECLARE Boolean choice to false

    DO
        OPENFILE Infile "cinema.txt" for READ and APPEND
        INITIALIZE i to 0
        CLEAR console screen
        DISPLAY "Please choose a movie to delete."
        DISPLAY "Movie List"
        DISPLAY "Name Price"

        IF Infile is open THEN
            WHILE NOT EOF Infile
                READFILE Infile, movie[i].name
                READFILE Infile, movie[i].price
                DISPLAY i+1, ".", movie[i].name
                DISPLAY FIXED, SETPRECISION to 2
                DISPLAY "RM", movie[i].price
                i++
                READFILE IGNORE newline character
            END WHILE
            DISPLAY new line
            CLOSEFILE Infile
        ELSE
            DISPLAY "Unable to open storing file!"
        END IF
    END DO
END FUNCTION

```

```

    DISPLAY "Which one you want to delete? (Type MovieNo to Delete) : "
    READ dlno

    IF dlno <= I AND dlno >0 THEN
        INITIALIZE choice to true

    ELSE IF dlno > i OR dlno <= 0 THEN
        DISPLAY "Invalid input. Press ENTER to select again!"
        READ input stream
        READ input stream (let user to press ENTER)
    END IF

    WHILE NOT choice
    END WHILE

    CALCULATE dlno -= 1

    OPENFILE Outfile "cinema.txt" for TRUNCATE

    WHILE a < i - 1
        DECLARE Integer pos to find movie[dlno].name exist inside movie[a].name
        IF pos found THEN
            WHILE a < i - 1
                REPLACE movie[a].name to movie[a+1].name
                REPLACE movie[a].price to movie[a+1].price

                IF a != 0
                    WRITEFILE Outfile, new line
                    WRITEFILE Outfile, movie[a].name
                    WRITEFILE Outfile, movie[a].price
                    INITIALIZE pos to not find
                    a++
                END WHILE
            END WHILE

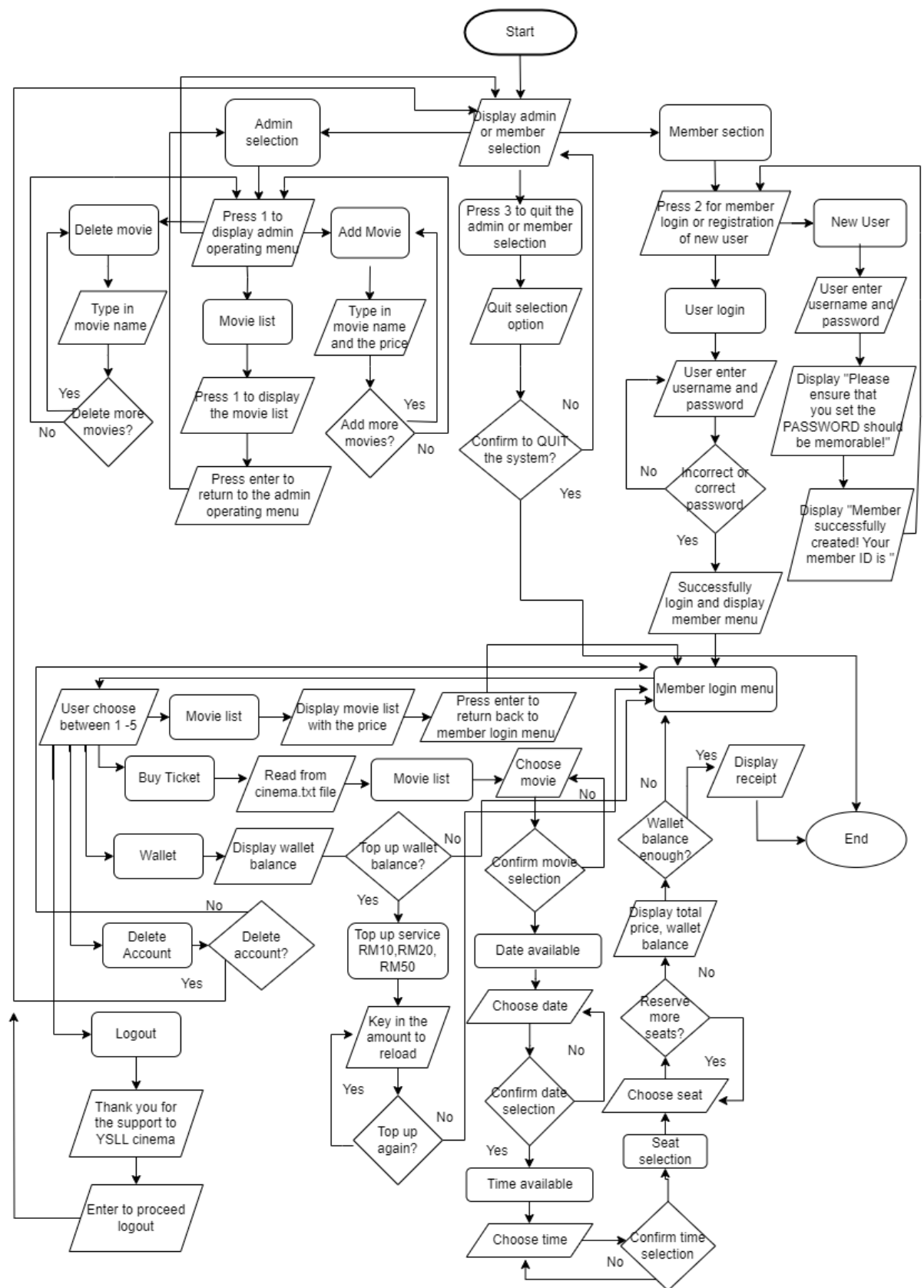
        ELSE IF pos not found AND a != i -1 THEN

```

```
        IF a != 0
            WRITEFILE Outfile, new line
            WRITEFILE Outfile, movie[a].name
            WRITEFILE Outfile, movie[a].price
            a++
        END IF
    END WHILE

    CLOSEFILE Outfile
    DISPLAY "Press ENTER to return back to Operating Menu."
    READ input stream
    READ input stream (let user to press ENTER)
    CLEAR console screen
END
```

v. Flowchart



vi. Test Case

1. Add movie

Movie List	
Name	Price
1. Spiderman: No Way Home	RM21.40
2. Demon Slayer: Kimetsu No Yaiba	RM23.00
3. John Wick: Chapter 4	RM18.70
4. The Conjuring: Chapter 3	RM17.00
5. Love Yourself: HYNH	RM24.00
6. Interstellar	RM20.70

Press ENTER to return back to Operating Menu.

Original

YSLC Cinema - Admin Operating Menu	
1. Movie List	
2. Add Movie	
3. Delete Movie	
4. Back	

Please input a selection: 2

Add Movie
Name: The Boy in the Striped Pajamas
Price: 20.8
Do you want to add more movie?(Y/N): y

Add Movie
Name: Tenet
Price: 20.2
Do you want to add more movie?(Y/N): n

Press ENTER to return back to Operating Menu.

Input

Movie List	
Name	Price
1. Spiderman: No Way Home	RM21.40
2. Demon Slayer: Kimetsu No Yaiba	RM23.00
3. John Wick: Chapter 4	RM18.70
4. The Conjuring: Chapter 3	RM17.00
5. Love Yourself: HYNH	RM24.00
6. Interstellar	RM20.70
7. The Boy in the Striped Pajamas	RM20.80
8. Tenet	RM20.20

Press ENTER to return back to Operating Menu.

Output

2. Delete movie

Movie List	
Name	Price
1. Spiderman: No Way Home	RM21.40
2. Demon Slayer: Kimetsu No Yaiba	RM23.00
3. John Wick: Chapter 4	RM18.70
4. The Conjuring: Chapter 3	RM17.00
5. Love Yourself: HYNH	RM24.00
6. Interstellar	RM20.70
7. The Boy in the Striped Pajamas	RM20.80
8. Tenet	RM20.20

Press ENTER to return back to Operating Menu.

Original

Please choose a movie to delete.	
Movie List	
Name	Price
1. Spiderman: No Way Home	RM21.40
2. Demon Slayer: Kimetsu No Yaiba	RM23.00
3. John Wick: Chapter 4	RM18.70
4. The Conjuring: Chapter 3	RM17.00
5. Love Yourself: HYNH	RM24.00
6. Interstellar	RM20.70
7. The Boy in the Striped Pajamas	RM20.80
8. Tenet	RM20.20

Which one you want to delete? (Type MovieNo to Delete) : 4
Press ENTER to return back to Operating Menu.

Input

Movie List	
Name	Price
1. Spiderman: No Way Home	RM21.40
2. Demon Slayer: Kimetsu No Yaiba	RM23.00
3. John Wick: Chapter 4	RM18.70
4. Love Yourself: HYNH	RM24.00
5. Interstellar	RM20.70
6. The Boy in the Striped Pajamas	RM20.80
7. Tenet	RM20.20

Press ENTER to return back to Operating Menu.

Output

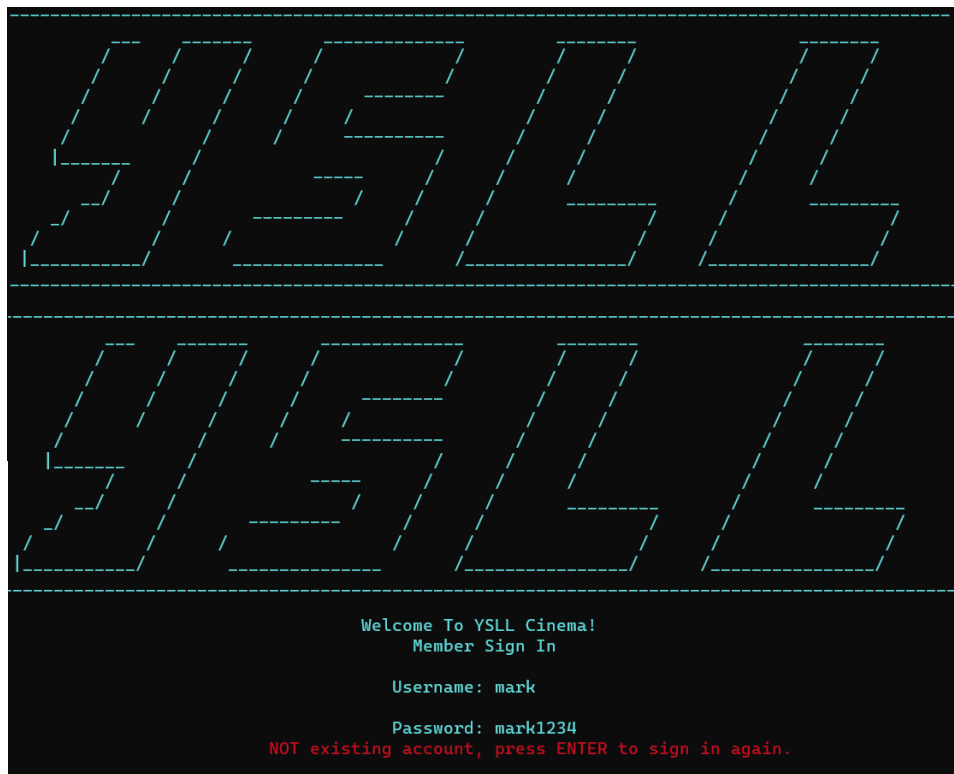
3. User register

Register
Successful

Repeat Username (Invalid)

Password Format (Invalid)

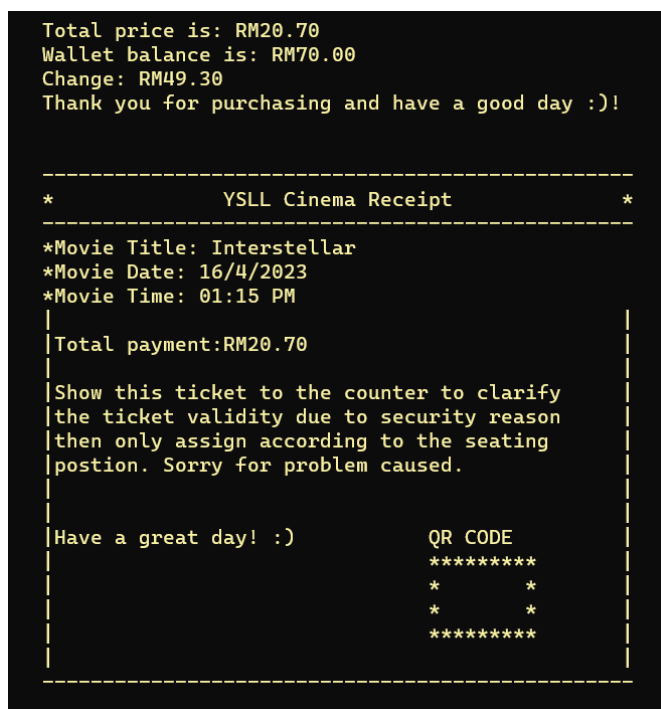
4. User Login



Login Successful

Login
Unsuccessful

5. Buy Ticket



Receipt

6. Top Up Wallet

```
Total price is: RM37.40
Wallet balance is: RM0.00
Error: Insufficient amount.
Payment Unsuccessful, please top up WALLET before transaction.
Press ENTER return back to Member Menu.
```

Insufficient amount
to buy ticket

Top Up Service
RM10
RM20
RM50
RM100

Choose Top Up Value

```
Please key in a value to TOPUP: 20
Would you like to TOP UP again? (Y/N): y

Please key in a value to TOPUP: 50
Would you like to TOP UP again? (Y/N): n
Press ENTER to return back to Member MENU.
```

```
|-----|
|      Wallet      |
|-----|
|      Balance = RM70.00      |
|-----|
| Would you like to top up your wallet?(Y/N): |
```

Wallet after Top Up

7. Seat

```
|-----|
```

```
                SCREEN
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
Enter column number (1-20): 1
```

```
Enter row number (1-10): 2
```

```
Seat is already reserved. Press ENTER to select again!
```

Seat unavailable.

Can choose multiple seat.

8. Delete Account

```
|-----|
|  YSLL Cinema - Member Login Menu  |
|-----|
| 1. Movie List                      |
| 2. Buy Ticket                      |
| 3. Wallet                          |
| 4. Delete Account                  |
| 5. Logout                          |
|-----|
Please input a selection: 4

Do you want to delete your current account? (Y/N): y
Account deleted. Press ENTER auto logout...
```

Delete current
account

Account deleted, can't login anymore

```
-----
|  YSLL  |
|-----|
Welcome To YSLL Cinema!
Member Sign In

Username: x1zz

Password: x1zz0808
NOT existing account, press ENTER to sign in again.
```

vii. Source Code

```
#include<iostream>
#include<string>
#include<fstream>
#include<iomanip>
#include<cctype>
#include<cstring>
#define _CRT_SECURE_NO_WARNINGS //VS secure version to use strcpy function
using namespace std;

//global variable
const int arraySize = 1000;
const int pwSize = 9;

const int rows = 10;
const int columns = 20;
int seats[rows][columns] = { 0 };

struct movieType {
    string name;
    double price;
};

struct memberSign {
    string name;
    char password[pwSize];
};

//function prototype
void admb();
void admMenu(int&);
void read();
```

```

void add();
void dlt();
void mbMenu();
void mbSignup();
bool testNum(char [], int);
void mbLogin(int&);
void mbLoginMenu(int&);
void mbbuyticket(int, int, double&);
void print_seating_chart();
double calculate_total_cost(int, int);
bool reserve_seat(int, int);
void mbwallet(double&);
void mbcancelacc(int, bool&);

int main() {

    admb();

    cin.get();
    return 0;
}

//big menu
void admb() {

    int choiceA, num;
    bool choice = false;
    char quitChoice;

    do {
        system("color 0d");
        cout << "\t|-----|" << endl;
        cout << "\t|  YSLL Cinema    |" << endl;
    } while (true);
}

```

```

cout << "\t|-----|" << endl;
cout << "\t|1. Admin Mode    |" << endl;
cout << "\t|2. Member Mode    |" << endl;
cout << "\t|3. Quit            |" << endl;
cout << "\t|-----|" << endl;
cout << "        Please input a selection: ";
cin >> choiceA;

if (choiceA == 1) { //admin mode
    do {
        system("cls");
        admMenu(num);
        if (num == 1) { //movie list
            read();
        }
        else if (num == 2) { //add movie
            add();
        }
        else if (num == 3) { //delete movie
            dlt();
        }
        else if (num > 4 || num < 0) { //num invalid
            cout << "        Invalid input. Please press ENTER to
select again!" << endl;
            cin.get();
            cin.get();
        }
        else if (num == 4) //back
            choice = false;
    } while (num > 0 && num != 4);
    system("cls");
}

else if (choiceA == 2) { //member mode

```

```

        system("cls");
        mbMenu();
    }
    else if (choiceA == 3) { //quit
        cout << "Confirm to QUIT the system?(Y/N): ";
        cin >> quitChoice;
        while (quitChoice != 'Y' && quitChoice != 'y' && quitChoice !=
'n' && quitChoice != 'N') { //only accept Y|y or N|n
            cout << "Invalid input. Please re-type again! (Y-Yes, N-
No): ";
            cin >> quitChoice;
        }
        if(quitChoice == 'Y' || quitChoice == 'y') //confirm to quit
system
            choice = true;
            system("cls");
        }
        else { //choiceA invalid
again!";
            cout << "        Invalid input. Please press ENTER to select
again!";
            cin.get();
            cin.get();
            system("cls");
        }
    } while (!choice);
}

```

//member sign in OR sign up

```
void mbMenu() {
```

```
    int choiceB, a; //'a' value will initialize in mbLogin function
```

```
    bool choice = false;
```

```
    do {
```

```
        system("color 0b");
```

```

cout << "\t|-----|" << endl;
cout << "\t|  YSLL Cinema - NEW USER OR EXIST?    |" << endl;
cout << "\t|-----|" << endl;
cout << "\t|1. New Register                            |" << endl;
cout << "\t|2. User Login                                |" << endl;
cout << "\t|-----|" << endl;

cout << "    Please input a selection: ";
cin >> choiceB;

if (choiceB == 1) { //register
    system("cls");
    mbSignup();
    system("cls");
}

else if (choiceB == 2) { //user login
    system("cls");
    mbLogin(a); //'a' value return by reference in mbLogin
function
    mbLoginMenu(a); //'a' value bring into mbLoginMenu function
    choice = true;
}

else { //choiceB invalid
    cout << "Invalid input. Please press ENTER to select again!"
<< endl;

    cin.get();
    cin.get();
    system("cls");
}

} while (!choice);

```

```
}
```

```
//member sign up
```

```
void mbSignup() {
```

```
    int i = 0;
```

```
    bool valid = false;
```

```
    memberSign member[arraySize];
```

```
    ofstream outfile("member.txt", ios::app);
```

```
    ifstream infile("member.txt", ios::app);
```

```
    while (!infile.eof()) { //read in member file
```

```
        getline(infile, member[i].name);
```

```
        infile.getline(member[i].password, pwSize);
```

```
        i++;
```

```
    }
```

```
    infile.close();
```

```
    //'i' now updated to the new array position
```

```
    system("color 0b");
```

```
    cout << " -----  
-----" << endl;
```

```
    cout << "          _____  
          " << endl;
```

```
    cout << "          /      /      /      /      /  
/          /      /      " << endl;
```

```
    cout << "          /      /      /      /      /  
/          /      /      " << endl;
```

```
    cout << "          /      /      /      /      /      -----  
/          /      /      " << endl;
```

```
    cout << "          /      /      /      /      /      /      /  
/          /      " << endl;
```

```
    cout << "          /      /      /      /      /      /      /  
/          /      " << endl;
```

```
    cout << "          |_____|      /      /      /  
/          /      " << endl;
```



```

    cout << "          /      /          -----      /      /      /
/      /          " << endl;
    cout << "      _/      /          /      /      /
_____      /      _____      " << endl;
    cout << "      _/      /          -----      /      /
/      /          /      " << endl;
    cout << "      /      /      /          /      /
/      /          /      " << endl;
    cout << "      |_____/_      _____
/_____/_      /_____/_      " << endl;
    cout << "
_____
_____ " << endl;

    cout << endl;
    cout << "\\t\\t\\t\\t\\tWelcome To YSLL Cinema!" << endl;
    cout << "\\t\\t\\t\\t\\t New Member Sign Up" << endl;
    cout << "\\t\\t \\033[31m*Please ensure that you set the PASSWORD
should be memorable!*\\033[0m" << endl;
    cout << endl;

    bool usernamevalid = false;
    int a;

    cout << "\\t\\t\\t\\t\\tUsername: ";
    cin.ignore();
    getline(cin, member[i].name); //input username

    do{
        for (a = 0; a < i; a++) { //check username not repeat then proceed
to set password
            if (member[i].name != member[a].name)
                usernamevalid = true;
        }

        for (a = 0; a < i; a++) { //check username repeat or not, if
repeat, cannot use second time

```

```

        if(member[i].name == member[a].name) { //a=3 i=4
            cout << "\t\t\t\t \033[031mThis username has been created
by other user.\033[0m" << endl;

            cout << "\t\t\t\t\t\033[031mPlease re-type other
username!\033[0m" << endl;

            cout << endl;
            cout << "\t\t\t\t\t\tUsername: ";
            getline(cin, member[i].name);
            cin.ignore();
            usernamevalid = false;
        }
    }
} while (!usernamevalid);

outfile << endl;
outfile << member[i].name << endl; //username save in file
cout << endl;
do {
    cout << "\t\t\t\t\t\t\033[34m*L-Letter; N-Number*\033[0m" << endl;
    cout << "\t\t\t\t\t\t\033[34m(Format - LLLLNNNN)\033[0m" << endl;
    cout << "\t\t\t\t\t\tPassword: ";
    cin.getline(member[i].password, pwSize); //input password

    if (testNum(member[i].password, pwSize)) { //identify password
format using bool function
        outfile << member[i].password; //password save in file
        cout << endl;
        cout << "\t\t\t\tMember sucessfully created! Your member ID is "
<< "\033[32m" << member[i].name << "0" << i+1 << "\033[0m" << "." << endl;
        cout << "\t\t\t\t\t\t Press ENTER to return back to menu.";
        cin.get();
        valid = true;
    }
    else { //wrong password format
        cout << endl;
    }
}

```

```
        cout << "\t\t\t\t\033[031mFormat incorrect! Please set the  
PASSWORD again.\033[0m" << endl;
```

```
    }
```

```
    } while (!valid);
```

```
    outfile.close();
```

```
}
```

```
//identify password format
```

```
bool testNum(char memberpw[], int pwSize) {
```

```
    int count;
```

```
    for (count = 0; count < 4; count++) {
```

```
        if (!isalpha(memberpw[count]))
```

```
            return false;
```

```
    }
```

```
    for (count = 4; count < pwSize - 1; count++) {
```

```
        if (!isdigit(memberpw[count]))
```

```
            return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
//member login
```

```
void mbLogin(int& a) { //initialize 'a' same as the variable 'lgno'  
(member login)
```

```
    a = 0; //'a' define as the array location of current user login
```

```
    int i = 0, lgno = 0;
```

```
    bool valid = false;
```

```
    memberSign member[arraySize], loginmb[arraySize];
```

```

ifstream infile("member.txt", ios::app);

while (!infile.eof()) {
    getline(infile, member[i].name);
    infile.getline(member[i].password, pwSize);
    i++; //'i' can define as the total of existing member
}
infile.close();

do {
    system("color 0b");
    cout << " -----" << endl;
    -----" << endl;
    cout << "
    cout << " / / / / / / / /
/ / / " << endl;
    cout << " / / / / / / / /
/ / / " << endl;
    cout << " / / / / / ----- /
/ / / " << endl;
    cout << " / / / / / / / /
/ / / " << endl;
    cout << " / / / / / ----- /
/ / / " << endl;
    cout << " |_____/ / / / /
/ / / " << endl;
    cout << " / / / ----- / /
/ / / " << endl;
    cout << " / / / / / / / /
/ / / " << endl;
    cout << " |_____/_____/_____/
/_____/_____/_____/ " << endl;

```

```

        cout << "
_____
_____
" << endl;

        cout << endl;
        cout << "\t\t\t\t\t Welcome To YSLL Cinema!" << endl;
        cout << "\t\t\t\t\t Member Sign In" << endl;
        cout << endl;
        cin.ignore();
        cout << "\t\t\t\t\tUsername: ";
        getline(cin, loginmb[lgno].name); //input login name
        cout << endl;
        cout << "\t\t\t\t\tPassword: ";
        cin.getline(loginmb[lgno].password, pwSize); //input login
password

        for (a = 0; a < i; a++) { //identify whether the login name and
password exist inside the member file or not

                if ((member[a].name == loginmb[lgno].name) &&
(strncmp(member[a].password, loginmb[lgno].password) == 0)) { //name and
password also same, exist acc

                        cout << "\t\t\t\t\tYou're successfully log in! Press ENTER to
MEMBER MENU." << endl;

                                cin.get();
                                system("cls");
                                valid = true;

                                break; //once find the exist acc, get inside IF function,
need to stop the FOR loop to invoid execute a++

                }

        }

        if ((member[a].name != loginmb[lgno].name) ||
(strncmp(member[a].password, loginmb[lgno].password) == -1)) { //invalid
account

                cout << "\t\t\t\t\t \033[31mNOT existing account, press ENTER
to sign in again.\033[0m" << endl;

                        cin.get();
                        system("cls");

```

```

    }
    } while (!valid);

}

//member login menu
void mbLoginMenu(int& a) {

    int choiceC, current_idno = a; //'current_idno' initialize to 'a'
    double balance = 0; //declare member wallet balance
    movieType movie[arraySize];

    bool loginChoice = false, auto_logout = false, buyticketChoice =
false;

    do {
        system("color 0b");
        cout << "\t|-----|" << endl;
        cout << "\t|  YSLL Cinema - Member Login Menu      |" << endl;
        cout << "\t|-----|" << endl;
        cout << "\t|1. Movie List                                |" << endl;
        cout << "\t|2. Buy Ticket                                    |" << endl;
        cout << "\t|3. Wallet                                           |" << endl;
        cout << "\t|4. Delete Account                                |" << endl;
        cout << "\t|5. Logout                                           |" << endl;
        cout << "\t|-----|" << endl;
        cout << "\tPlease input a selection: ";
        cin >> choiceC;
        cout << endl;

        if (choiceC == 1) { //movie list
            read();
        }
    }

```

```

else if (choiceC == 2) { //buy ticket
    do {
        system("cls");
        ifstream infile("cinema.txt", ios::app);
        int i = 0, movieNo;

        cout << "\t1. Buy Ticket" << endl;
        cout << "\t|-----|
-----|" << endl;
        cout << "\t|\t\t\tMovie List\t\t\t|" << endl;
        cout << "\t|-----|
-----|" << endl;
        cout << "\t| Name\t\t\t\t\t | Price |" << endl;
        cout << "\t|-----|
-----|" << endl;

        if (infile.is_open()) {
            while (!infile.eof()) { //read in file
                getline(infile, movie[i].name);
                infile >> movie[i].price;
                cout << "\t| " << i + 1 << ". " << movie[i].name
<< endl;

                cout << fixed << setprecision(2);
                cout << "\t|\t\t\t\t\t | RM" <<
movie[i].price << " |" << endl;
                i++; //total movie number
                infile.ignore();
            }
            cout << "\t|-----|
-----|" << endl;
            infile.close();
        }

        cout << "\tChoose a movie: ";
        cin >> movieNo;
    } while (choiceC != 3);
}

```

```

        if (movieNo <= i) { //choose movieNo within the movie
number
            mbbuyticket(movieNo, i, balance); //balance renew
            system("cls");
            buyticketChoice = true;
        }
        else { //movieNo chosen out of bound
            cout << "\tInvalid input. Press ENTER to re-select
again!" << endl;
            cin.get();
            cin.get();
        }
    } while (!buyticketChoice);

}

else if (choiceC == 3) { //wallet
    mbwallet(balance);
    system("cls");
}

else if (choiceC == 4) { //delete account
    mbcancelacc(current_idno, auto_logout); //'current_idno' = 'a'
value bring into mbcancelacc function
    system("cls");
    if (auto_logout) //'auto_logout' value takeout from
mbcancelacc function to do IF function
        loginChoice = true; //quit from this whole loop
    }

else if (choiceC == 5) { //logout
    cout << "\tThank for your support to our YSLL Cinema!" <<
endl;

    cout << "\tPress ENTER to proceed logout." << endl;
    cin.get();
    cin.get();
    system("cls");
    loginChoice = true;
}

```



```

    }

    } while(!loginChoice);
}

//member buy ticket
void mbbuyticket(int movieNo, int i, double& balance) { //i = total movie
number

    movieType movie[arraySize];
    system("cls");
    system("color 0d");

    int date, time, numSeats = 0;
    double payment, totalPayment, change;

    while (true) {
        cout << "\t-----" << endl;
        cout << "\t|          Date Available          |" << endl;
        cout << "\t-----" << endl;
        cout << "\t|          1.16/4/2023          |" << endl;
        cout << "\t|          2.17/4/2023          |" << endl;
        cout << "\t|          3.18/4/2023          |" << endl;
        cout << "\t|          4.19/4/2023          |" << endl;
        cout << "\t|          5.20/4/2023          |" << endl;
        cout << "\t-----" << endl;
        cout << "\tSelect a date: ";
        cin >> date;

        if (date >= 1 && date <= 5) {
            break;
        }
        else {
            cout << "Please enter a number between 1 to 5 only\n\n";

```

```

    }
}
cout << endl;
system("CLS");

while (true) {
    cout << "You have selected the date" << date << "." << endl;
    switch (date) {
        case 1:
            cout << "16/4/2023";
            break;
        case 2:
            cout << "17/4/2023";
            break;
        case 3:
            cout << "18/4/2023";
            break;
        case 4:
            cout << "19/4/2023";
        case 5:
            cout << "20/4/2023";
            break;
    }
    cout << endl;

    cout << "Do you want to change your selection? (Y/N): ";
    char choice;
    cin >> choice;
    system("CLS");

    if (choice == 'Y' || choice == 'y') {
        while (true) {
            cout << "\t-----" << endl;

```

```

        cout << "\t|          Date Available          |" << endl;
        cout << "\t-----" << endl;
        cout << "\t|          1.16/4/2023          |" << endl;
        cout << "\t|          2.17/4/2023          |" << endl;
        cout << "\t|          3.18/4/2023          |" << endl;
        cout << "\t|          4.19/4/2023          |" << endl;
        cout << "\t|          5.20/4/2023          |" << endl;
        cout << "\t-----" << endl;
        cout << "\tSelect a date: ";
        cin >> date;

        if (date >= 1 && date <= 5) {
            break;
        }
        else {
            cout << "Please enter a number between 1 to 5 only" <<
endl << endl;
        }
    }
    cout << endl;
    system("CLS");
}
else if (choice == 'N' || choice == 'n') {
    break;
}
else {
    cout << "Invalid input. Please enter Y or N only." << endl <<
endl;
}
}

cout << "You have selected date " << date << "." << endl;
switch (date) {
case 1:

```

```

        cout << "16/4/2023";
        break;
case 2:
        cout << "17/4/2023";
        break;
case 3:
        cout << "18/4/2023";
        break;
case 4:
        cout << "19/4/2023";
case 5:
        cout << "20/4/2023";
        break;
}
cout << endl;
system("CLS");

//Choosing time section
system("color 0e");
while (true) {
    cout << "\t-----" << endl;
    cout << "\t|          Time available          |" << endl;
    cout << "\t-----" << endl;
    cout << "\t|          1. 10:00 AM          |" << endl;
    cout << "\t|          2. 01:15 PM          |" << endl;
    cout << "\t|          3. 01:45 PM          |" << endl;
    cout << "\t|          4. 03:45 PM          |" << endl;
    cout << "\t|          5. 04:15 PM          |" << endl;
    cout << "\t|          6. 06:45 PM          |" << endl;
    cout << "\t|          7. 09:15 PM          |" << endl;
    cout << "\t-----" << endl;
    cout << "\tSelect a time: ";
    cin >> time;
}

```

```
    if (time >= 1 && time <= 7) {
        break;
    }
    else {
        cout << "Please enter a number between 1 to 7 only\n\n";
    }
}
cout << endl;
system("CLS");

while (true) {
    cout << "You have selected the time" << time << "." << endl;
    switch (time) {
        case 1:
            cout << "10:00 AM";
            break;
        case 2:
            cout << "01:15 PM";
            break;
        case 3:
            cout << "01:45 PM";
            break;
        case 4:
            cout << "03:45 PM";
            break;
        case 5:
            cout << "04:15 PM";
            break;
        case 6:
            cout << "06:45 PM";
            break;
        case 7:
            cout << "09:15 PM";
```

```

        break;
    }
    cout << endl;

    cout << "Do you want to change your selection? (Y/N): ";
    char choice;
    cin >> choice;
    system("CLS");

    if (choice == 'Y' || choice == 'y') {
        while (true) {
            cout << "\t-----" << endl;
            cout << "\t|          Time available      |" << endl;
            cout << "\t-----" << endl;
            cout << "\t|          1. 10:00 AM          |" << endl;
            cout << "\t|          2. 01:15 PM          |" << endl;
            cout << "\t|          3. 01:45 PM          |" << endl;
            cout << "\t|          4. 03:45 PM          |" << endl;
            cout << "\t|          5. 04:15 PM          |" << endl;
            cout << "\t|          6. 06:45 PM          |" << endl;
            cout << "\t|          7. 09:15 PM          |" << endl;
            cout << "\t-----" << endl;
            cout << "\tSelect a time: ";
            cin >> time;

            if (time >= 1 && time <= 7) {
                break;
            }
            else {
                cout << "Please enter a number between 1 to 7 only" <<
endl << endl;
            }
        }
        cout << endl;
    }

```

```
        system("CLS");
    }
    else if (choice == 'N' || choice == 'n') {
        break;
    }
    else {
        cout << "Invalid input. Please enter Y or N only." << endl <<
endl;
    }
}
```

```
cout << "You have selected time " << time << "." << endl;
switch (time) {
case 1:
    cout << "10.00 AM";
    break;
case 2:
    cout << "01:15 PM";
    break;
case 3:
    cout << "01:45 PM";
    break;
case 4:
    cout << "03:45 PM";
    break;
case 5:
    cout << "04:15 PM";
    break;
case 6:
    cout << "06:45 PM";
    break;
case 7:
    cout << "09:15 PM";
    break;
```

```

}
cout << endl;
system("CLS");

cout << endl;

char choice = 'y';
do {
    // print seating chart
    system("color 0e");
    cout << "\t\t-----" << endl;
    cout << "\t\t|                                     |" << endl;
    cout << "\t\t-----" << endl;
    cout << "\t\t\t\t\tSCREEN\n" << endl;

    print_seating_chart();

    // get user input
    cout << endl;
    cout << "\t\tEnter column number (1-" << columns << "): ";
    int seat;
    cin >> seat;
    cout << "\t\tEnter row number (1-" << rows << "): ";
    int row;
    cin >> row;

    // validate input
    if (row < 1 || row > rows || seat < 1 || seat > columns) {
        cout << "\t\tInvalid input. Press ENTER to select again." <<
endl;

        cin.get();
        cin.get();
        system("cls");
        continue; //skip below and loop again
    }
} while (choice == 'y');
}

```



```

    }

    // reserve seat
    bool reserved = reserve_seat(row, seat);
    if (reserved) {
        cout << "\t\tSeat reserved." << endl;
        numSeats++;
    }
    else {
        cout << "\t\tSeat is already reserved. Press ENTER to select
again!" << endl;
        cout << endl;
        cin.get();
        cin.get();
        system("cls");
        continue;
    }

    // ask whether the user wants to reserve more seats or not
    cout << "\t\tDo you want to reserve more seats? (Y/N): ";
    cin >> choice;
    system("cls");
} while (choice == 'Y' || choice == 'y');

// calculate total cost
totalPayment = calculate_total_cost(numSeats, movieNo);

// display the total payment for the user
cout << endl << endl;
cout << "\t\tTotal price is: RM" << totalPayment << endl;
cout << "\t\tWallet balance is: RM" << balance << endl;

```

```

// calculate change for the extra payment
change = balance - totalPayment;
bool payment_successful = true;

// output change of extra payment or error message
if (change >= 0) {
    balance = change;
    cout << "\t\tChange: RM" << change << endl;
    cout << "\t\tThank you for purchasing and have a good day :)" <<
endl;
    cout << endl;
}
else {
    cout << "\t\t\033[31mError: Insufficient amount.\033[0m" << endl;
    cout << "\t\t\033[31mPayment Unsuccessful, please top up WALLET
before transaction.\033[0m" << endl;
    cout << "\t\t\033[31mPress ENTER return back to Member
Menu.\033[0m" << endl;
    cin.get();
    cin.get();
    payment_successful = false;
}

cout << endl;

while (payment_successful) {
    cout << "\t\t-----" <<
endl;
    cout << "\t\t*                YSLL Cinema Receipt                *" <<
endl;
    cout << "\t\t-----" <<
endl;
    cout << "\t\t*Movie Title: ";

    ifstream infile("cinema.txt");

```

```
while (!infile.eof()) {
    for (int a = 0; a < i; a++) {
        getline(infile, movie[a].name);
        infile >> movie[a].price;
        infile.ignore();
    }
}
infile.close();

cout << movie[movieNo - 1].name;

cout << endl;

cout << "\t\t*Movie Date: ";
switch (date) {
case 1:
    cout << "16/4/2023";
    break;
case 2:
    cout << "17/4/2023";
    break;
case 3:
    cout << "18/4/2023";
    break;
case 4:
    cout << "19/4/2023";
case 5:
    cout << "20/4/2023";
    break;
}
cout << endl;

cout << "\t\t*Movie Time: ";
```

```

switch (time) {
case 1:
    cout << "10.00 AM";
    break;
case 2:
    cout << "01:15 PM";
    break;
case 3:
    cout << "01:45 PM";
    break;
case 4:
    cout << "03:45 PM";
    break;
case 5:
    cout << "04:15 PM";
    break;
case 6:
    cout << "06:45 PM";
    break;
case 7:
    cout << "09:15 PM";
    break;
}
cout << endl;

cout << "\t\t|\t\t\t\t\t\t|" << endl;
cout << "\t\t|" << "Total payment:RM" << totalPayment <<
"\t\t\t\t|" << endl;
cout << "\t\t|\t\t\t\t\t\t|" << endl;
cout << "\t\t|Show this ticket to the counter to clarify\t|" <<
endl;
cout << "\t\t|the ticket validity due to security reason\t|" <<
endl;
cout << "\t\t|then only assign according to the seating\t|" <<
endl;

```



```
}
```

```
//calculate total payment of the chosen movie with chosen how many seats
```

```
double calculate_total_cost(int numSeats, int movieNo) {
```

```
    movieType movie[arraySize];
```

```
    ifstream infile("cinema.txt");
```

```
    int i = 0;
```

```
    while (!infile.eof()) {
```

```
        infile.ignore();
```

```
        getline(infile, movie[i].name);
```

```
        infile >> movie[i].price;
```

```
        i++;
```

```
    }
```

```
    return numSeats * movie[movieNo-1].price;
```

```
}
```

```
//chosen seat reserved
```

```
bool reserve_seat(int row, int seat) {
```

```
    if (seats[row - 1][seat - 1] == 0) {
```

```
        seats[row - 1][seat - 1] = 1;
```

```
        return true;
```

```
    }
```

```
    else {
```

```
        return false;
```

```
    }
```

```
}
```

```
//member wallet
```

```
void mbwallet(double& balance) {
```

```
    int topup;
```

```
    char topupChoice;
```

```
    bool topupexit = false;
```

```

system("cls");
cout << "\t|-----|" << endl;
cout << "\t|          Wallet          |" << endl;
cout << "\t|-----|" << endl;
cout << fixed << setprecision(2);
cout << "\t    Balance = RM" << balance << endl;
cout << "\t|-----|" << endl;
cout << "\tWould you like to top up your wallet?(Y/N): ";
cin >> topupChoice;

while (topupChoice != 'Y' && topupChoice != 'y' && topupChoice != 'N'
&& topupChoice != 'n'){
    cout << "\tInvalid input. Please re-type again! (Y-Yes, N-No): ";
    cin >> topupChoice;
}

if (topupChoice == 'Y' || topupChoice == 'y') {

    system("cls");
    cout << "\t|-----|" << endl;
    cout << "\t| Top Up Service |" << endl;
    cout << "\t|-----|" << endl;
    cout << "\t|    RM10    |" << endl;
    cout << "\t|    RM20    |" << endl;
    cout << "\t|    RM50    |" << endl;
    cout << "\t|   RM100   |" << endl;
    cout << "\t|-----|" << endl;
    do {
        cout << endl;
        cout << "\tPlease key in a value to TOPUP: ";
        cin >> topup;

        switch (topup) {

```

```

        case 10: {
            balance += topup;
            break;
        }
        case 20: {
            balance += topup;
            break;
        }
        case 50: {
            balance += topup;
            break;
        }
        case 100: {
            balance += topup;
            break;
        }
        default: {
            cout << "\tInvalid input. Please re-type TOPUP value!" <<
endl;

            continue;
        }
    }

    cout << "\tWould you like to TOP UP again? (Y/N): ";
    cin >> topupChoice;

    while (topupChoice != 'Y' && topupChoice != 'y' &&
topupChoice != 'N' && topupChoice != 'n') {
        cout << "\tInvalid input. Please re-type again! (Y-Yes, N-
No): ";

        cin >> topupChoice;
    }
    //if (topupChoice == 'Y' || topupChoice == 'y') loop again

```



```

        if (topupChoice == 'N' || topupChoice == 'n') {
            cout << "\tPress ENTER to return back to Member MENU." <<
endl;

            cin.get();
            cin.get();
            topupexit = true;
        }

```

```

    } while (!topupexit);

```

```

}

```

```

else if (topupChoice == 'N' || topupChoice == 'n') {
    cout << "\tPress ENTER to return back to Member MENU." << endl;
    cin.get();
    cin.get();
}
}

```

```

//member delete account

```

```

void mbcancelacc(int current_idno, bool& auto_logout) { //'current_idno'
come from 'a'

```

```

    int i = 0;

```

```

    char deleteChoice;

```

```

    memberSign member[arraySize];

```

```

    ifstream infile("member.txt", ios::app);

```

```

    while (!infile.eof()) {

```

```

        getline(infile, member[i].name);

```

```

        infile.getline(member[i].password, pwSize);

```

```

        i++;

```

```

    }

```

```

    infile.close();

```

```

cout << "\tDo you want to delete your current account? (Y/N): ";
cin >> deleteChoice;

while (deleteChoice != 'Y' && deleteChoice != 'y' && deleteChoice !=
'N' && deleteChoice != 'n') {
    cout << "\t\033[031mInvalid input. Please re-type again! (Y-Yes,
N-No): \033[0m";
    cin >> deleteChoice;
}

if (deleteChoice == 'Y' || deleteChoice == 'y') {

    ofstream outfile("member.txt", ios::trunc); //rewrite the file

    for (int a = 0; a < i-1; a++) {
        if ((member[a].name == member[current_idno].name) &&
(strcmp(member[a].password, member[current_idno].password) == 0)) {

            member[a].name = member[a + 1].name; //replace
            strcpy_s(member[a].password, member[a + 1].password);
//replace

            if (a != 0)
                outfile << endl;
            outfile << member[a].name << endl; //write in file
            outfile << member[a].password; //write in file
        }

        else if ((member[a].name != member[current_idno].name) ||
(strcmp(member[a].password, member[current_idno].password) == -1)) {

            if (a != 0)
                outfile << endl;
            outfile << member[a].name << endl; //write back to file

```

```

        outfile << member[a].password; //write back to file
    }
}
outfile.close();
cout << "\tAccount deleted. Press ENTER auto logout..." << endl;
cin.get();
cin.get();
auto_logout = true;
}
else if (deleteChoice == 'N' || deleteChoice == 'n') {
    cout << "\tPress ENTER return back to Member MENU." << endl;
    cin.get();
    cin.get();
}

}

//admin menu
void admMenu(int& choice) {

    system("color 0a");
    cout << "\t|-----|" << endl;
    cout << "\t|  YSLL Cinema - Admin Operating Menu  |" << endl;
    cout << "\t|-----|" << endl;
    cout << "\t|1. Movie List                                |" << endl;
    cout << "\t|2. Add Movie                                |" << endl;
    cout << "\t|3. Delete Movie                             |" << endl;
    cout << "\t|4. Back                                    |" << endl;
    cout << "\t|-----|" << endl;
    cout << "\t    Please input a selection: ";
    cin >> choice;
}

```

```

//view movie
void read() {

    ifstream infile("cinema.txt", ios::app);
    movieType movie[arraySize];
    int i = 0;

    system("cls");
    cout << "\t|-----|"
<< endl;
    cout << "\t|\t\t\tMovie List\t\t\t|" << endl;
    cout << "\t|-----|"
<< endl;
    cout << "\t| Name\t\t\t\t\t | Price   |" << endl;
    cout << "\t|-----|"
<< endl;

    if (infile.is_open()) {
        while (!infile.eof()) {
            getline(infile, movie[i].name);
            infile >> movie[i].price;
            cout << "\t| " << i+1 << ". " << movie[i].name << endl;
            cout << fixed << setprecision(2);
            cout << "\t|\t\t\t\t\t | RM" << movie[i].price << " |" <<
endl;
            i++;
            infile.ignore();
        }
        cout << "\t|-----"
-|" << endl;
        infile.close();
    }
    else
        cout << "\tUnable to open storing file!" << endl;
}

```

```

        cout << "\tPress ENTER to return back to Operating Menu." << endl;
        cin.get();
        cin.get();
        system("cls");
    }

```

```

//add movie

```

```

void add() {

    ifstream infile("cinema.txt", ios::app);
    ofstream outfile("cinema.txt", ios::app);
    movieType movie[arraySize];
    char addMovie;

    do {
        int i = 0;
        while (!infile.eof()) {
            getline(infile, movie[i].name);
            infile >> movie[i].price;
            i++;
            infile.ignore();
        }
        cout << endl;
        cout << "\tAdd Movie" << endl;
        cout << "\tName: ";
        outfile << endl;
        cin.ignore();
        getline(cin, movie[i].name);
        outfile << movie[i].name << endl;
        cout << "\tPrice: ";
        cin >> movie[i].price;
        outfile << movie[i].price;
    } while (addMovie != 'q');
}

```

```

        cout << "\tDo you want to add more movie?(Y/N): ";
        cin >> addMovie;

        while (addMovie != 'Y' && addMovie != 'y' && addMovie != 'N' &&
addMovie != 'n') {
            cout << "\tInvalid input. Please type again (Y-Yes, N-No): ";
            cin >> addMovie;
        }

    } while (addMovie == 'Y' || addMovie == 'y');

    infile.close();
    outfile.close();

    cout << endl;
    cout << "\tPress ENTER to return back to Operating Menu." << endl;
    cin.get();
    cin.get();
    system("cls");
}

//delete movie
void dlt() {
    movieType movie[arraySize];
    int i, a = 0, dlno;
    bool choice = false;

    do {
        ifstream infile("cinema.txt", ios::app);
        i = 0;
        system("cls");

        cout << "\tPlease choose a movie to delete." << endl;

```

```

        cout << "\t|-----|";
-|" << endl;

        cout << "\t|\t\t\tMovie List\t\t\t|" << endl;
        cout << "\t|-----|";
-|" << endl;

        cout << "\t| Name\t\t\t\t\t | Price |" << endl;
        cout << "\t|-----|";
-|" << endl;


    if (infile.is_open()) {
        while (!infile.eof()) {
            getline(infile, movie[i].name);
            infile >> movie[i].price;
            cout << "\t| " << i + 1 << ". " << movie[i].name << endl;
            cout << fixed << setprecision(2);
            cout << "\t|\t\t\t\t\t | RM" << movie[i].price << "
|" << endl;

            i++;
            infile.ignore();
        }

        cout << "\t|-----|";
-----|" << endl;

        cout << endl;
        infile.close();
    }
    else
        cout << "Unable to open storing file!" << endl;

//i is the total number of movie
//i - 1 is the index of last movie
cout << "\tWhich one you want to delete? (Type MovieNo to
Delete) : ";

cin >> dlno;


if (dlno <= i && dlno > 0)
    choice = true;

```

```

        else if (dlno > i || dlno <= 0) {
            cout << "\tInvalid input. Press ENTER to select again!" <<
endl;

            cin.get();
            cin.get();
        }
    } while (!choice);

    dlno -= 1; //dlno array or index position

    ofstream outfile("cinema.txt", ios::trunc); //rewrite the file

    while (a < i - 1) { //delete movie will be deducted from file, index
also -1

        int pos = movie[a].name.find(movie[dlno].name, 0);

        if (pos != string::npos) {

            while (a < i - 1) {

                movie[a].name = movie[a + 1].name;
                movie[a].price = movie[a + 1].price;

                if (a != 0)
                    outfile << endl;
                outfile << movie[a].name << endl;
                outfile << movie[a].price;

                pos = movie[a].name.find(movie[dlno].name, pos +
movie[dlno].name.length());
                a++;
            }
        }
    }

```



```
        else if (pos == string::npos && (a != i - 1)) {
            if (a != 0)
                outfile << endl;
            outfile << movie[a].name << endl;
            outfile << movie[a].price;
            a++;
        }
    }

    outfile.close();
    cout << "\tPress ENTER to return back to Operating Menu." << endl;
    cin.get();
    cin.get();
    system("cls");
}
```